

■ React Essentials

Components · JSX · Props · State · Events · Lifecycle

Easy English Quick-Reference Guide

1 What is React?

React is a popular JavaScript library for building user interfaces. It lets you break your web page into small, reusable pieces called **components**. Each component manages its own look and data.

2 Components

What is a Component?

- A component is a small, reusable piece of the UI (like a button, card, or header).
- In React, we mostly use Function Components — plain JavaScript functions that return JSX.
- Component names must start with a capital letter, e.g. MyButton.

Component Lifecycle (Function Components)

- Mount → The component appears on the screen for the first time.
- Re-render → React re-draws the component when its state or props change.
- Unmount → The component is removed from the screen.
- Use the `useEffect` hook to run code at each lifecycle stage.

The key Prop

- When you render a list, each item needs a unique key prop.
- React uses key to track which items changed, were added, or removed.
- Always use a stable, unique value (e.g. an id) — avoid using the array index.
- Example:

3 JSX

What is JSX?

- JSX looks like HTML but lives inside JavaScript files.
- React converts JSX into real browser elements behind the scenes.
- Use `{ }` curly braces to embed any JavaScript expression inside JSX.
- Example: `Hello, {userName}!`

- Every JSX element must have ONE root element (or use a Fragment `<>...`).

4 Props

Passing Data with Props

- Props (properties) let a parent component send data to a child component.
- They work like HTML attributes:
- Inside the child, receive them as a function parameter: `function Card(props) { ... }`
- Or use object destructuring for cleaner code: `function Card({ title }) { ... }`

The children Prop

- Anything placed between the opening and closing tags is available as `props.children`.
- Useful for wrapper/layout components.
- Example: Some content → inside Card, use `{props.children}`

Passing Functions as Props

- You can pass a function as a prop value, just like any other value.
- Useful for passing event handlers from parent to child.
- Example:

5 State

Managing State with `useState`

- State is data that can change over time and causes the component to re-render.
- Import and call `useState`: `const [count, setCount] = useState(0);`
- `count` is the current value; `setCount` is the function to update it.
- Always call the setter function to change state — never mutate directly.
- React re-renders the component every time the state is updated.

6 Events

Handling Events

- React uses camelCase event names: `onClick`, `onChange`, `onSubmit`, etc.
- Pass a function reference — do NOT call it: `onClick={handleClick}` (not `onClick={handleClick()}`).
- The event object is passed automatically to your handler function.
- To update the UI in response to an event, update state inside the handler.

7 Conditional Rendering

Showing Content Conditionally

- Use the ternary operator: `{ isLoggedIn ? : }`
- Use the `&&` shortcut: `{ isLoggedIn && }`
- Or store JSX in a variable and render it conditionally.

8 Dynamic Styles & Rendering Lists

CSS Styling — Dynamic Classes & Styles

- Apply inline styles as a JS object: `style={{ color: isActive ? 'green' : 'red' }}`
- Toggle class names dynamically: `className={isActive ? 'active' : ''}`
- For complex class toggling, consider libraries like `clsx`.

Rendering List Data

- Use the `Array .map()` method to turn an array into JSX elements.
- Each element must have a unique `key` prop.
- Example: `items.map(item => {item.name})`

9 Dynamic HTML Attributes & Image Files

Setting Attributes Dynamically

- Any HTML attribute can be set from a variable: `<img src={imageUrl} altText={...} />`
- For local image files in Vite/CRA, import them first: `import logo from './logo.png'`
- Then use the imported variable as the `src`: `<img src={logo} />`

10 Code Examples (GitHub Links)

function-component-using-outputting-dynamic-values

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/03-using-outputting-dynamic-values>

function-component-html-attributes-dynamically-image-files

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/04-html-attributes-dynamically-image-files>

function-component-passing-data-by-props

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/05-cmp-props>

function-component-passing-data-by-props-and-object-destructuring

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/06-alternative-props-syntax>

function-component-children-prop

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/09-composition-children-prop>

function-component-passing-functions-as-values-and-children

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/11-passing-functions-as-values>

function-component-with-event-example

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/13-how-not-to-update-ui>

function-component-with-usestate

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/14-managing-state>

function-component-rendering-content-conditionally

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/16-rendering-content-conditionally>

function-component-css-styling-dynamic-styles

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/17-css-styling-dynamic-styles>

function-component-outputting-list-data-demo-project-1

- <https://github.com/academind/react-complete-guide-course-resources/tree/main/code/03%20React%20Essentials/18-outputting-list-data>